JAVA UNIT-3 MATERIAL

Ch. No.	Topic	Details	Marks
3	Exception Handling, Threading and Streams (Input and Output)	• Introduction to exception handling (try,catch, finally, throw, throws) • Creating user defined exception class • Thread and its life cycle (Thread States) • Thread class and its methods • Synchronization in multiple threads (Multithreading) • Deamon thread • Non-Daemon thread • Stream and its types (Input, Output, Character, Byte) • File and Random access file • Reading and writing through Character Stream classes (File Reader, Buffered Reader, File Writer, Buffered Writer) • Reading and writing through byte stream class (Inputstream, FileInputStream, DataInputStream, OutputStream, FileOutputStream, DataOutputStream) • StreamTokenizer Class • Piped Streams, Bridge Classes: InputStreamReader, OutputStreamWriter • ObjectInputStream, ObjectOutputStream	14

Q-1 what is Exception Handling?

Ans:

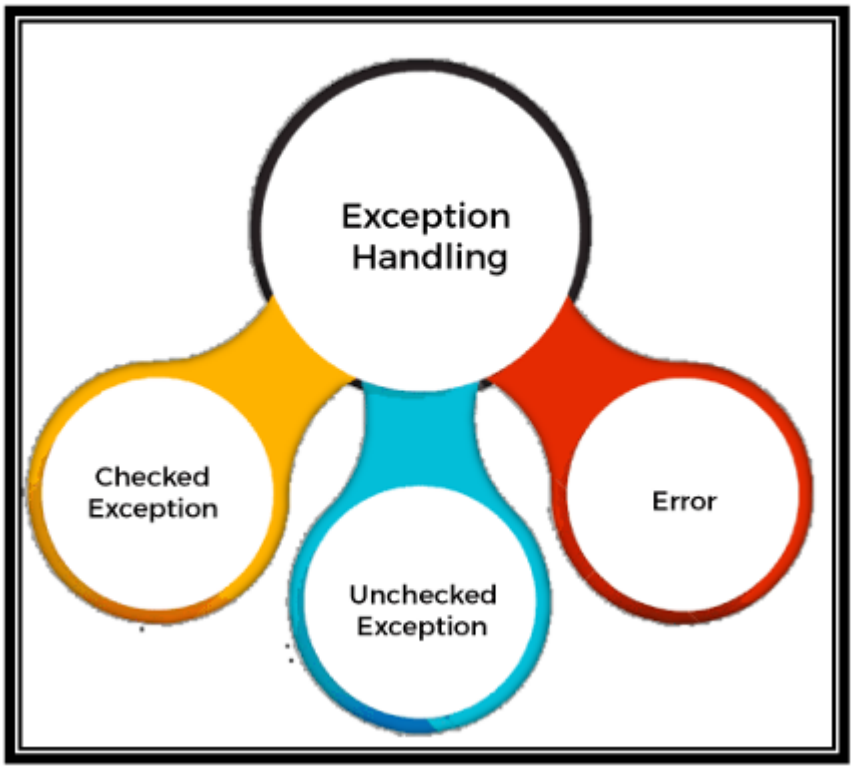
👉 The main benefit of exception handling is that it keeps the program running normally.

👉 When an exception happens, it breaks or disturbs the normal flow of the program.

👉 That is why we use exception handling — to manage the error and continue the program without stopping.

👉 The Exception Handling in Java is one of the powerful mechanism to handle the runtime errors so that the normal flow of the application can be maintained.

Types of Java Exceptions



✅ **try**

👉 Used to write code where error can happen

👉 Must be followed by catch or finally

👉 Cannot use alone

✅ **catch**

👉 Used to handle (fix) the error

👉 Always comes after **try**

👉 Cannot use alone

✅ **finally**

👉 Used to run important code

👉 Runs **always** (error happens or not)

✅ **throw**

👉 Used to **manually create and throw an error**

✅ **throws**

👉 Used in method to **tell that error may happen**

👉 It does NOT throw error, just gives warning

```
class exc
{
 public static void main(String args[])
 {
  try
  {
   int a = 5 / 0;   // error (divide by zero)
  }
  catch (ArithmeticException e)
  {
   System.out.println(e);  // prints error
  }
  finally
  {
   System.out.println("hi"); // always runs
  }
 }
}
```

🔥 **Output:**

```
java.lang.ArithmeticException: / by zero
hi
```

🧠 **Super Easy Memory Trick**

👉 try = check code

👉 catch = handle error

👉 finally = always run

👉 throw = create error

👉 throws = declare error

✅ **Checked Exception**

👉 Checked at **compile time** (before running)
👉 Comes from **Throwable**, but NOT **RuntimeException**
👉 Example: **IOException**

✅ **Checked Exception**	✅ **UnChecked Exception**
👉 Checked at **compile time** (before running)	👉 **UnChecked Exception** Checked at **runtime** (during execution)
👉 Comes from **Throwable**, but NOT **RuntimeException**	👉 Comes from **RuntimeException**
👉 Example: **IOException**	👉 Example: **ArithmeticException**

throw	throws
Used to actually throw an error	**Used to declare that an error may happen**
Inside method / block	**In method signature**
Only one exception at a time	**Can declare multiple exceptions**

🔥 **Super Easy Memory:**

👉 **Checked = compile time**
👉 **Unchecked = runtime**

MCQ	
1) Which is the superclass of all the errors and exceptions?	Throwable
2) Which keyword is used to write the code that contains error or exception?	try
3) Which keyword is used to write the code that must be compulsory executed?	finally
4) .....Exception is declared at compile time	Checked Exception

Q-2 what is User Defined Exception? How to create it

Ans:

👉 **Creating our own Exception is known as custom exception or user-defined exception. Basically, Java custom exceptions are used to customize the exception according to user need.**

👉 **This can be done by extending the class Exception**

```
class Test {
  public static void main(String[] args) {
    try {
      throw new MyException(400);
    } catch (MyException e) {
      System.out.println(e);
    }
  }
}


class MyException extends Exception {
```

```
  int n;

  MyException(int n) {
    this.n = n;
  }

  public String toString() {
    return "Status: " + n;
  }
}
```

MCQ	
1) Creating the customized exception is known as	User defined exception
2) User defined exception should extend .....Class	Exception

Q-3 What is Thread?

👉 A thread is a small part of a program that runs step-by-step.
👉 One program (process) can have many threads running at the same time.
👉 Each thread has its own program counter (it tracks what to run next).

🔄 Life Cycle of Thread (States)

👉 In Java, a thread can be in these states:

1. New
2. Active
3. Blocked / Waiting
4. Timed Waiting

5. Terminated

✅ 1) New

👉 When a thread is created, it is in new state
👉 It has not started running yet

⚡ 2) Active

👉 When we call start() → thread becomes active

👉 Active has 2 parts:

🔷 Runnable

👉 Thread is ready to run
👉 It may run anytime
👉 Thread scheduler decides when it runs

🔷 Running

👉 Thread gets CPU
👉 Now it is actually running

⛔ 3) Blocked / Waiting

👉 Thread is temporarily not working

👉 Example:
👉 Thread A wants to use printer
👉 Thread B is already using printer
👉 So thread A must wait → blocked state

⏳ 4) Timed Waiting

👉 Thread waits for fixed time (not forever)

👉 Problem: sometimes thread waits forever → called starvation

👉 Solution: timed waiting

👉 Example:
👉 Using sleep() method → thread waits for some time
👉 After time ends → thread starts again

OR

⏳ 4) Timed Waiting

👉 Thread waits for some fixed time
👉 It does not wait forever

👉 If a thread waits forever → it is called starvation

👉 To avoid this, we use timed waiting

👉 Example:
👉 Using sleep() → thread stops for some time
👉 After time is over → thread starts again

🔥 Super Short:

👉 Timed Waiting = wait for limited time

❌ 5) Terminated

👉 Thread is finished

👉 Two ways:

- ✅ Normal → work completed
- ❌ Abnormal → error (like exception)

🔥 Super Easy Memory Trick

👉 New → created
👉 Active → running / ready
👉 Blocked → waiting for resource
👉 Timed → waiting with time limit
👉 Terminated → finished

🧠 **One-Line Revision**

👉 **Thread = small task inside program**
👉 **Life cycle = states from start → run → wait → end**

MCQ	
1) .....is a sequential flow of the control within the program	Thread
2) There can be more than one thread inside the process (T/F)?	True
3) The active state of thread contains ..... and .....states	Runnable, Running
4) Which process occurs when some unusual events such as an unhandled exception or segmentation fault?	Abnormal Termination

Q-4 **What is Thread Class? Explain its methods.**

👉 **Thread class is used to create and control threads in Java**
👉 **It gives methods to start, run, stop, and manage threads**

🔧 **Ways to Create Thread**

👉 **3 ways:**

1. **Extend Thread Class**
2. **Implement Runnable**
3. **Create Thread object directly**

✅ **1) Extending Thread Class**

👉 **Create a class that extends Thread**
👉 **Override run() method**
👉 **Call start() to run thread**

💻 **Your Code (Simple + Fixed)**

```
class M extends Thread {
  public void run() {
    System.out.println("Running");
  }

  public static void main(String[] args) {
    M t1 = new M();
    t1.start();
  }
}
```

🔍 **What Happens**

👉 **start() is called**
👉 **JVM calls run() method**
👉 **Output: Running**

2) Implementing Runnable Interface:

👉 **Create Runnable implementer which implements run()**

👉 **Create the object of Thread Class**

👉 **Pass the instance of class in Thread class**

👉**Runnable interface have only one method named run().**

```
class M implements Runnable {
  public void run() {
    System.out.println("Running");
  }
  public static void main(String[] args) {
    M obj = new M();
    Thread t1 = new Thread(obj);
    t1.start();
  }
}
```

3) Explicitly creating thread object using Thread Class:

👉 **If we don't extend the Thread class, then object of thread class is created explicitly.**

EXAMPLE:

```
class M {
  public static void main(String[] args) {
    Thread t1 = new Thread("First Thread");
    t1.start();
    System.out.println("Thread Name: " + t1.getName());
  }
}
```

Thread Methods Table

Method	Description
void run()	Used to perform action for thread
void start()	Starts the execution of thread. JVM calls run() on thread
public String getName()	It is used to get the name of thread
public void setName(String)	It is used to change or set the name of the given thread
public int getPriority()	It is used to get the priority of the thread
public int setPriority()	It is used to set the priority of the thread
public void suspend()	It is used to suspend the given thread
public void resume()	It is used to resume the suspended thread
sleep(time)	Pauses the thread for some time
yield()	Gives chance to other threads to run
isDaemon()	Checks if thread is daemon or not

```
class Th extends Thread {
  public void run() {
    for (int i = 1; i <= 5; i++) {
      try {
        Thread.sleep(1000);
      } catch (InterruptedException e) {
        System.out.println(e);
      }
      System.out.println(i);
    }
  }
```

```
  public static void main(String[] args) {
    Th t1 = new Th();

    t1.setName("My Thread");
    t1.setPriority(6);

    System.out.println("Thread Name: " + t1.getName());
    System.out.println("Thread Priority: " + t1.getPriority());
    System.out.println("Daemon Thread: " + t1.isDaemon());

    t1.start();
  }
}
```

🔥 **Output (Example)**

```
Thread Name: My Thread
Thread Priority: 6
Daemon Thread: false
1
2
3
4
5
```

🔍 **What this fixed code does**

👉 Sets thread name and priority
👉 Prints thread details
👉 Runs loop from 1 to 5
👉 Waits 1 second between each number

MCQ	
1) Thread can be created by implementing .....interface	Runnable
2) Which method of thread class moves the thread from new state to active state?	start()
3) Which method of thread class is called on the start () of thread?	run()
4) Runnable interface have only 1 method. Which is that?	run()
5) Which method causes currently executing thread object to temporarily pause and allow other threads to execute	yield()

Q-5) Write a short note on Thread Synchronization?

OR

Write a short note on Multithreading

Ans:

✅ **Thread Synchronization (Simple + Smart Notes)**

🔷 **What is Synchronization?**

- Synchronization means **controlling multiple threads** when they use the **same resource**.
- It makes sure that **only one thread works at a time** on that resource.

👉 Simple line:
Only one thread can use shared data at a time.

🔷 **Why we use Synchronization?**

Synchronization is used to avoid problems:

1. **Thread interference**
    - When many threads run together, they may **mix their work**.

2. **Consistency problem**
    - Data may become **wrong or corrupted**.

👉 Simple idea:
Without synchronization → data becomes messy
With synchronization → data stays correct

🔷 Types of Synchronization in Java

You can do synchronization in 3 ways:

1. **Synchronized Method**
2. **Synchronized Block**
3. **Static Synchronization**

❌ Example WITHOUT Synchronization (Problem)

What happens?

- Two threads (t1 and t2) use same object
- Both run at the same time
- Output gets mixed

Code idea:

- t1 prints table of 5
- t2 prints table of 10

- 🔷 **Code (Without Synchronization)**
-
```
class table {
 void printtable(int n) {
  for(int i=1;i<=5;i++) {
   System.out.println(n*i);
   try {
    Thread.sleep(500);
   } catch(Exception e) {
    System.out.println(e);
   }
  }
 }
}
```

- 👉 Threads:
- ```
  class mythread1 extends Thread {
   table t;
   mythread1(table t) { this.t = t; }
   public void run() {
    t.printtable(5);
   }
  }

  class mythread2 extends Thread {
   table t;
   mythread2(table t) { this.t = t; }
   public void run() {
    t.printtable(10);
   }
  }
  ```
- 👉 Main:
- ```
  class synch {
   public static void main(String args[]) {
    table o = new table();
    mythread1 t1 = new mythread1(o);
    mythread2 t2 = new mythread2(o);
    t1.start();
    t2.start();
   }
  }
  ```
- 👉 Problem:
  Output will look like:
- ```
  5
  10
  10
  15
  20
  ...
  ```

✅ **Example WITH Synchronization (Solution)**

What changed?

- We added **synchronized keyword**
- Now only **one thread runs at a time**

👉 Result:

- First thread completes fully
- Then second thread starts

🔷 **Code (With Synchronization)**

```
class table {
 synchronized void printtable(int n) {
```
```

```
 for(int i=1;i<=5;i++) {
  System.out.println(n*i);
  try {
   Thread.sleep(500);
  } catch(Exception e) {
   System.out.println(e);
  }
 }
}
}
```

👉 Threads:

```
class mythread1 extends Thread {
 table t;
 mythread1(table t) { this.t = t; }
 public void run() {
  t.printtable(5);
 }
}

class mythread2 extends Thread {
 table t;
 mythread2(table t) { this.t = t; }
 public void run() {
  t.printtable(5);
 }
}
```

👉 Main:

```
class synch {
 public static void main(String args[]) {
  table o = new table();
  mythread1 t1 = new mythread1(o);
  mythread2 t2 = new mythread2(o);
  t1.start();
  t2.start();
 }
}
```

👉 Output:

```
5
10
15
20
25
(then next thread starts)
```

👉 Solution:
Synchronization = control system

✓ One thread at a time
✓ No data mixing
✓ Safe and correct output

Q- 6 : **Write a short note on Daemon and Non-Daemon Thread**

Ans:

🔷 **Daemon Thread (Background Thread)**

- A **Daemon thread** is a **low-priority thread**
- It runs in the **background**
- It performs **support tasks** (like cleaning memory)

👉 Example:

- Garbage Collection (automatic memory cleaning)

👉 Simple meaning:
Helper thread that supports main work

✓ **Key Points**

- Runs in **background**
- Works for **user threads**
- Called **service provider thread**
- JVM **does NOT wait** for it to finish
- Stops automatically when user threads end

🔷 Non-Daemon Thread (User Thread)

- A **Non-Daemon thread** is a **normal thread**
- It runs in the **foreground**
- It performs **main/important tasks**

👉 Simple meaning:
Main working thread of program

✓ Key Points

- Runs in **foreground**
- Does **actual important work**
- JVM **waits** until it finishes
- Program ends only after these threads finish

🔥 Easy Comparison Table

Feature	Daemon Thread	Non-Daemon Thread
Created by	JVM	Application (Programmer)
Work type	Background work	Main work
Priority	Low	High
JVM behavior	JVM does NOT wait	JVM waits to finish
Task type	Not critical	Critical
Life	Depends on user thread	Independent

Daemon Threads	User Threads (Non-daemon)
Daemon threads are created by JVM	User threads are created by an application itself
JVM does not wait for its execution	JVM waits until the execution completes
Low Priority threads	High priority threads
Used for background tasks(not critical)	Used for foreground tasks(critical)
Life is dependent on user threads	Life is independent

Q-7) what is Stream? Explain types

Ans :

🔷 **Definition (Very Simple)**

- A **Stream** is a **flow of data**
- It moves data from **source → program → destination**

👉 Simple idea:
Stream = Data pipeline (like water flow)

🔷 **Important Points**

- Stream is a **sequence of data**
- In Java, data moves in **bytes (small units)**
- Java uses streams to make **Input/Output (I/O) fast**
- All stream classes are inside **java.io package**

🔷 **Types of Streams (Basic)**

1️⃣ InputStream

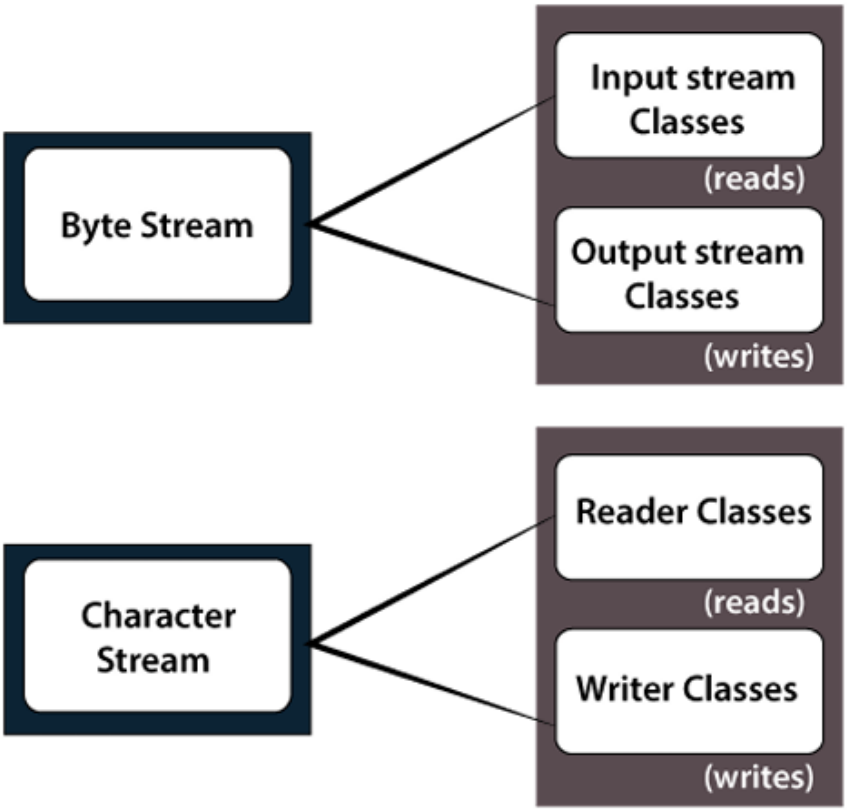
- Used to **read data**
- Data comes **from source → program**

👉 Example:
Reading file, keyboard input

2️⃣ OutputStream

- Used to **write data**
- Data goes **from program → destination**

👉 Example:
Writing file, displaying output



Brief classification of I/O streams

🔷 **Types Based on Data Handling**

1⃣ **Byte Stream**

✓ **What it does:**

- Works with **8-bit data (byte)**
- Reads/writes **raw data**

✓ **Used for:**

- Images
- Videos
- Audio
- Files

👉 Simple idea:
Handles all types of data

2⃣ Character Stream

✓ **What it does:**

- Works with **16-bit Unicode**
- Handles **text data only**

✓ **Used for:**

- Reading/writing **characters & strings**

👉 Simple idea:
Only for text (letters, words)

🔥 **Difference Table (Very Easy)**

Feature	Byte Stream	Character Stream
Data size	8-bit	16-bit (Unicode)
Data type	Any type (image, video, etc.)	Only text
Reading style	Byte by byte	Character by character
Also called	Data streams	Reader/Writer streams

🧠 **Brain Boost Trick**

👉 **Byte Stream = All data (raw)**
👉 **Character Stream = Only text**

MCQ	
1) .....is sequence of data	Stream
2) Which package contains all the class for input and output operations?	java.io package
3) Which streams are used to perform input and output for 8-bit bytes	Byte Stream
4) Which stream are tied to data type?	Character Stream

Q-8) what is File Class?

Ans:

1) Create New File

✓ Key Point:

- Method: createNewFile()
- Returns:
  - true → file created
  - false → already exists

```
File f = new File("D:\\hel.txt");
if(f.createNewFile())
 System.out.println("Created");
else
 System.out.println("Not Created");
```

---

2) Get File Information

✓ Methods:

- getName() → file name

- getAbsolutePath() → full path
- length() → file size
- exists() → check file

```
if(f.exists()) {
 System.out.println(f.getName());
 System.out.println(f.getAbsolutePath());
 System.out.println(f.length());
}
```

---

3) Write into File

✓ Class Used:

- FileWriter

✓ Important:

- Must use close() after writing

```
FileWriter w = new FileWriter("d:\\hel.txt");
w.write("Hello");
w.close();
```

---

4) Read from File

✓ Class Used:

- Scanner

✓ Methods:

- hasNextLine()
- nextLine()

```
File f = new File("D:\\hel.txt");
Scanner s = new Scanner(f);
while(s.hasNextLine()) {
 String data = s.nextLine();
 System.out.println(data);
}
s.close();
```

---

5) Delete File

✓ **Method:**

- delete()

```
File f = new File("D:\\hel.txt");

if(f.delete())
 System.out.println("Deleted");
else
 System.out.println("Not Deleted");
```

🧠 **Brain Boost Summary**

👉 **File class = File manager in Java**

✓ Create file
✓ Check file
✓ Get info
✓ Write data
✓ Read data
✓ Delete file

MCQ	
1) Which method is used to create the file?	createNewFile()
2) createNewFile() must be used with .....exception	IOException
3) In order to write the data into file, which class is used?	FileWriter
4) Which class is used to read the data from the file?	Scanner
5) hasnextLine() and nextLine() are the methods of .....class	Scanner
6) .....method is used to delete the file	delete()

Q-9) What is Random Access File Class?

🔷 **Basic Idea**

- Normally, files are read **sequentially (one by one)**
- But sometimes we want to **jump directly to any position**

👉 That's where **RandomAccessFile** is used

🔷 **Definition (Simple)**

- **RandomAccessFile** allows you to:
  - **Read and write data at any position in a file**
  - **Directly jump to any location**

👉 Simple idea:
Like skipping to any scene in a video (no need to watch from start)

🔷 **Sequential vs Random Access**

✓ **Sequential Access**

- Data is read **one by one**
- Slow if file is large

👉 Example: reading file from start till end

✓ **Random Access**

- Data is accessed **directly**
- Fast and efficient

👉 Example: jump to 5th record directly

🔷 **Key Features of RandomAccessFile**

✓ Can **read + write both**
✓ Can **move pointer anywhere**

✓ No need to read full file

✓ Useful for **large files / databases**

🔷 Important Methods (Easy Table)

Method	Meaning (Simple)
close()	Close the file
getFilePointer()	Get current position
length()	Get file size
read()	Read one byte
read(byte[])	Read multiple bytes
seek(long n)	Move pointer to position
skipBytes(n)	Skip some bytes

🔷 Most Important Concept

👉 File Pointer

- It shows **current position in file**
- All read/write operations happen from this point

👉 Example:

- seek(0) → go to start
- seek(10) → go to 10th byte

🔷 Example Code (Simple Understanding)

```
import java.io.*;

class Random1 {
 public static void main(String args[]) {
  try {
   RandomAccessFile f = new RandomAccessFile("test1.txt", "rw");
```

```
  f.writeChar('k');    // write character
  f.writeInt(10);      // write integer
  f.writeDouble(10.2);  // write double

  f.seek(0); // go to start

  System.out.println(f.readChar());
  System.out.println(f.readInt());
  System.out.println(f.readDouble());

  f.close();
 }
 catch(IOException e) {
  System.out.println(e);
 }
}
}
```

🧠 **Brain Boost Summary**

👉 **RandomAccessFile = Jump anywhere in file**

✓ Read/write anywhere
✓ No need to follow order
✓ Uses **file pointer**
✓ Faster than sequential

⚡ **One-Line Revision**

- **Sequential → step by step**
- **RandomAccessFile → direct access (jump anywhere)**
- **seek() → move pointer**

MCQ	
1) Random Access is used to perform .....and .....operations both simultaneously	read,write
2) In random access file, the file can be moved to any location using .....method	seek()

Q-10) Write a short note on character stream class

Ans:

🔷 **What is Character Stream?**

- Character stream is used to **read and write text data**
- It works with **16-bit Unicode characters**
- It solves the problem of **byte streams (8-bit only)**

👉 Simple idea:
Character Stream = Special stream for text (letters, words)

🔷 **Why we use Character Stream?**

- Byte stream handles only **8-bit data**
- It is **not good for characters directly**
- Character stream handles **16-bit Unicode → better for text**

👉 Example:
Reading/writing text files like .txt

🔷 **Main Classes of Character Stream**

Class	Simple Meaning
FileReader	Read characters from file
FileWriter	Write characters to file

Class	Simple Meaning
BufferedReader	Read text faster (with buffer)
BufferedWriter	Write text faster (with buffer)

🔷 FileReader & FileWriter

✓ Key Points

- Used for **text files**
- No need to convert into byte array
- Can directly **read/write strings**

🔷 Important Methods

Method	Meaning
read()	Read one character
write()	Write data
flush()	Clear output buffer

🔷 Example (FileReader & FileWriter)

```
import java.io.*;

class ch1 {
 public static void main(String args[]) {
  try {
   FileWriter fw = new FileWriter("D:\\myf6.txt");
   fw.write("Hello");
   fw.close();

   FileReader fr = new FileReader("D:\\myf6.txt");
   int i;

   while((i = fr.read()) != -1) {
    System.out.print((char)i);
```

```
  }

  fr.close();
 }
 catch(IOException e) {
  System.out.println(e);
 }
}
}
```

🔷 BufferedReader & BufferedWriter

✓ Key Idea

- Works with **buffer (temporary storage)**
- Makes reading/writing **faster and efficient**

👉 Simple idea:
Buffered = faster performance

🔷 Features

- Reads/writes **large text efficiently**
- Works with **strings, arrays**

🔷 Example (Buffered Classes)

```
import java.io.*;

class ch1 {
 public static void main(String args[]) {
  try {
   FileWriter fw = new FileWriter("D:\\myf7.txt");
   BufferedWriter bw = new BufferedWriter(fw);

   bw.write("Hello");
   bw.close();

   FileReader fr = new FileReader("D:\\myf7.txt");
```

```
  BufferedReader br = new BufferedReader(fr);

  int i;
  while((i = br.read()) != -1) {
   System.out.print((char)i);
  }

  br.close();
 }
 catch(IOException e) {
  System.out.println(e);
 }
}
}
```

🧠 **Brain Boost Summary**

👉 **Character Stream = For text only (16-bit)**

✓ FileReader → read text
✓ FileWriter → write text
✓ BufferedReader → fast reading
✓ BufferedWriter → fast writing

⚡ **One-Line Revision**

- **Character Stream → Text handling (16-bit Unicode)**
- **FileReader/FileWriter → basic read/write**
- **Buffered → faster performance**

MCQ	
1) Character Stream class works on .....bits	16
2) BufferedReader inherits from	FileReader

Note :-

બફરનો ઉપયોગ શા માટે થાય છે?

તમે જે જાવા (Java) પ્રોગ્રામિંગની નોટ્સ વાંચી રહ્યા છો તેમાં BufferedReader અને BufferedWriter નો ઉલ્લેખ છે. તેનો ઉપયોગ નીચે મુજબ છે:

સ્પીડ વધારવા: હાર્ડ ડિસ્કમાંથી ડેટા (એક-એક અક્ષર) વાંચવાની કે લખવાની પ્રક્રિયા ખૂબ ધીમી હોય છે.

ડેટા ભેગો કરવા: બફરનો ઉપયોગ કરવાથી, કમ્પ્યુટર એકસાથે ઘણો બધો ડેટા મેમરીમાં (બફરમાં) વાંચી લે છે. પછી જ્યારે તમારા પ્રોગ્રામને જરૂર પડે ત્યારે તે બફરમાંથી ઝડપથી ડેટા આપે છે. આનાથી સિસ્ટમનું પર્ફોર્મન્સ ફાસ્ટ થાય છે.

Q-11) Write a short note on Byte stream class

Ans:

🔷 **What is Byte Stream?**

- Byte stream is used to **read and write data in bytes (8-bit)**
- It works with **raw data**

👉 Simple idea:
Byte Stream = handles all types of data (image, audio, video, file)

🔷 **Types of Byte Stream**

1⃣ InputStream (Read Data)

- Used to **read bytes**
- Source can be:
  - File
  - Console
  - Memory

👉 Note:

- It is **abstract class (cannot create object directly)**

🔷 Subclasses of InputStream

Class	Simple Meaning
BufferedInputStream	Read faster using buffer
ByteArrayInputStream	Read from byte array
DataInputStream	Read primitive data (int, float etc.)
FileInputStream	Read from file
FilterInputStream	Read from another stream
ObjectInputStream	Read objects
PipedInputStream	Read from connected stream

🔷 Important Methods (InputStream)

Method	Meaning
read()	Read one byte
flush()	Clear stream
close()	Close stream

2) **OutputStream (Write Data)**

- Used to **write bytes**
- Sends data to destination

👉 Example:
File, console, network

🔷 Subclasses of OutputStream

Class	Simple Meaning
BufferedOutputStream	Write faster using buffer
ByteArrayOutputStream	Write to byte array
DataOutputStream	Write primitive data
FileOutputStream	Write to file
FilterOutputStream	Write to another stream
ObjectOutputStream	Write objects
PipedOutputStream	Write to connected stream

🔷 Important Methods (OutputStream)

Method	Meaning
write()	Write byte
flush()	Clear output
close()	Close stream

🔷 FileInputStream & FileOutputStream

✓ FileInputStream

- Reads **byte data from file**
- Used for:
  - Images
  - Audio
  - Video

✓ FileOutputStream

- Writes **byte data to file**
- Can write:

- Numbers
- Characters
- Raw data

🔷 **Example (FileInputStream + FileOutputStream)**

```
import java.io.*;

class inp {
 public static void main(String args[]) {
  try {
   FileOutputStream f = new FileOutputStream("D:\\myf9.txt");
   f.write(10);

   FileInputStream f1 = new FileInputStream("D:\\myf9.txt");
   int i;

   while((i = f1.read()) != -1) {
    System.out.println(i);
   }
  }
  catch(IOException e) {
   System.out.println(e);
  }
 }
}
```

🔷 **DataInputStream & DataOutputStream**

✓ **Purpose**

- Used to **read/write primitive data types**
  (int, float, double, etc.)

👉 Simple idea:
Special streams for numbers and data types

🔷 **Example**

```
import java.io.*;

class da {
 public static void main(String args[]) throws IOException {

  FileOutputStream f = new FileOutputStream("D:\\myf10.txt");
  DataOutputStream f1 = new DataOutputStream(f);
  f1.write(10);

  FileInputStream f2 = new FileInputStream("D:\\myf10.txt");
  DataInputStream f3 = new DataInputStream(f2);

  int i;
  while((i = f3.read()) != -1) {
   System.out.println(i);
  }

  f1.close();
  f3.close();
 }
}
```

🔷 **Program: Copy One File to Another**

👉 Logic:

- Read from first file
- Write into second file

🔷 **Example**

```
import java.io.*;

class cop {
 public static void main(String args[]) throws IOException {

  FileInputStream f = new FileInputStream("D:\\myf1.txt");
  FileOutputStream f1 = new FileOutputStream("D:\\copy.txt");
```

```
 int i;
 while((i = f.read()) != -1) {
  f1.write(i);
 }

 f.close();
 f1.close();
}
}
```

🧠 **Brain Boost Summary**

👉 **Byte Stream = Raw data (8-bit)**

✓ InputStream → read data
✓ OutputStream → write data
✓ FileInputStream → read file
✓ FileOutputStream → write file
✓ DataInput/Output → handle numbers

⚡ **One-Line Revision**

- **Byte Stream → handles all data (8-bit)**
- **InputStream → read**
- **OutputStream → write**
- **FileInput/Output → file handling**
- **DataInput/Output → primitive data**

MCQ	
1) Byte stream class works on .....bits	8 ibts
2) Which subclass of byte stream class is used to read and write the primitive data types	DataInputStream and DataOutputStream

🔷 **What are Piped Streams?**

- **PipedInputStream** and **PipedOutputStream** are used to:
  - **send data from one stream to another**
  - usually used between **two threads**

👉 Simple idea:
One thread writes → another thread reads

🔷 **How it works?**

- **PipedOutputStream → writes data**
- **PipedInputStream → reads data**
- Both are **connected together**

👉 Like a pipe:
Data flows from output → input

🔷 **Important Point**

- They are connected using:

in.connect(out);

👉 Without connection → they will not work

🔷 **Key Features**

✓ Used for **communication between threads**
✓ Works **simultaneously (read + write)**
✓ Data flows like a **pipeline**

🔷 **Example (Simple Understanding)**

```
import java.io.*;

class pipe {
 public static void main(String args[]) {
```

```
 PipedOutputStream out = new PipedOutputStream();
 PipedInputStream in = new PipedInputStream();

 try {
  in.connect(out);   // connect both streams

  out.write(23);     // write data
  out.write(24);

  for(int i=0; i<2; i++) {
   System.out.println(in.read());  // read data
  }

 } catch(IOException e) {
  System.out.println(e);
 }
}
}
```

🔷 **Output**

```
23
24
```

🧠 **Brain Boost Summary**

👉 **Piped Streams = Data transfer between threads**

✓ Output stream → sends data
✓ Input stream → receives data
✓ Must be **connected**
✓ Works like a **pipe**

MCQ	
1) Which method is used to connect piped input stream with piped output stream	connect()
2) What is main Feature of piped stream?	Allows to read and write the data simultaneously

Q-14) Write a short note on Bridge Class

Ans:

🔷 **What is a Bridge Class?**

- A **Bridge Class** is used to **connect two different streams**
- It converts:
  - **Byte Stream ↔ Character Stream**

👉 Simple idea:
Bridge = Converter (changes data type)

🔷 **1) InputStreamReader**

✓ **What it does:**

- Converts **byte stream → character stream**
- Reads bytes and converts them into characters

👉 Simple:
FileInputStream → InputStreamReader → Characters

🔷 **Example**

```
import java.io.*;

class br {
 public static void main(String args[]) throws IOException {

  FileInputStream f = new FileInputStream("D:\\myf3.txt");
```

```
 InputStreamReader r = new InputStreamReader(f);

 int i;
 while((i = r.read()) != -1) {
  System.out.println((char)i);
 }

 r.close();
}
}
```

🔷 **2) OutputStreamWriter**

✔ **What it does:**

- Converts **character stream → byte stream**
- Encodes characters into bytes

👉 Simple:
Characters → OutputStreamWriter → FileOutputStream

🔷 **Example**

```
import java.io.*;

class br {
 public static void main(String args[]) throws IOException {

  FileOutputStream f = new FileOutputStream("D:\\myf100.txt");
  OutputStreamWriter w = new OutputStreamWriter(f);

  w.write("hello world");
  w.close();
 }
}
```

🔷 **3) ObjectInputStream & ObjectOutputStream**

✔ **What they do:**

- Used to **read/write objects and primitive data**
- Work with **serialization (store object)** and **deserialization (read object)**

✓ **Simple Meaning:**

- **ObjectOutputStream → write object/data**
- **ObjectInputStream → read object/data**

🔷 **Example**

```
import java.io.*;

class br {
 public static void main(String args[]) throws IOException {

  int i = 10;

  FileOutputStream f = new FileOutputStream("d:\\myfile101.txt");
  ObjectOutputStream o = new ObjectOutputStream(f);
  o.writeInt(i);

  FileInputStream f1 = new FileInputStream("d:\\myfile101.txt");
  ObjectInputStream o1 = new ObjectInputStream(f1);

  System.out.println("Integer: " + o1.readInt());

  f.close();
  f1.close();
 }
}
```

🧠 **Brain Boost Summary**

👉 **Bridge Class = Converter between streams**

✓ InputStreamReader → byte → character
✓ OutputStreamWriter → character → byte
✓ Object Streams → store & read objects

⚡ **One-Line Revision**

- **Bridge Class → connect byte & character streams**
- **InputStreamReader → read characters from bytes**
- **OutputStreamWriter → convert characters to bytes**
- **Object Streams → handle objects & data**

MCQ	
1) .....class is used to convert byte stream to character stream class?	InputStreamReader
2) Which class is used to convert character stream to byte stream class?	OutputStreamWriter